Synchronization

Week 9

Operating Systems Ch 6 (Synchronization Tools), Ch 7 (Synchronization Examples)

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

Background & Critical-Section Problem (Ch 6.1 – 6.2)

Cooperating Processes and Data Consistency

- **Cooperating process**: a process that can affect or be affected by the execution of other processes
- Sharing methods: **shared memory**, **message passing**, **logical address space sharing**
- **Concurrent access** to shared data can cause **data inconsistency**
- An **orderly execution** mechanism is needed to prevent this

Race Condition

A phenomenon where **the result depends on the execution order** when multiple processes access shared data concurrently

Example: When `count = 5`, Producer executes `count++` and Consumer executes `count--` simultaneously

```
T0: producer  register1 = count        {register1 = 5}
T1: producer  register1 = register1+1  {register1 = 6}
T2: consumer  register2 = count        {register2 = 5}
T3: consumer  register2 = register2-1  {register2 = 4}
T4: producer  count = register1        {count = 6}
T5: consumer  count = register2        {count = 4}  ← incorrect result!
```

- The correct result is `count = 5`, but depending on the interleaving order, it can become 4 or 6
- Solution: protect the region that accesses shared data (**Critical Section**)

Race Condition – Kernel Examples

- A race condition can occur on the `next_available_pid` variable during **fork()**
  - Two processes calling fork() simultaneously may be assigned the same PID
- A race condition can occur when modifying the **open file list**
- Also occurs in **memory allocation**, **process list management**, **interrupt handling**, etc.
- Kernel developers must prevent such race conditions

Critical-Section Problem

Each process has a **critical section** where shared data is modified

```
while (true) {
    /* entry section */
        /* critical section */
    /* exit section */
        /* remainder section */
}
```

3 Requirements:

1. **Mutual Exclusion** – only one process can enter the critical section at a time
2. **Progress** – if the critical section is empty, a process must be selected from those not in the remainder section, and this selection cannot be postponed indefinitely
3. **Bounded Waiting** – after a process requests entry, there must be an upper bound on the number of times other processes can enter the critical section before the request is granted

Preemptive vs Nonpreemptive Kernel

- **Nonpreemptive kernel**: does not preempt a process running in kernel mode
  - Race conditions on kernel data are inherently avoided
  - However, responsiveness may suffer
- **Preemptive kernel**: preemption is possible even in kernel mode
  - More responsive and suitable for real-time programming
  - Careful design is needed to prevent race conditions in SMP environments
- **Single processor**: can be simply solved by disabling interrupts
- **Multiprocessor**: disabling interrupts is costly, so other methods are needed

Part 2

Peterson's Solution (Ch 6.3)

Peterson's Solution

A **software-based** solution for 2 processes (P0, P1)

Shared variables: `int turn;` and `boolean flag[2];`

```
/* Process Pi (i = 0 or 1, j = 1 - i) */
while (true) {
    flag[i] = true;         /* I want to enter */
    turn = j;               /* Yield to the other */
    while (flag[j] && turn == j)
        ;                   /* Wait if the other wants in and it's their turn */

    /* critical section */

    flag[i] = false;        /* I am done */

    /* remainder section */
}
```

- `turn` : indicates whose turn it is
- `flag[i]` : indicates that process i wants to enter the critical section

Peterson's Solution – Correctness Proof

1. Mutual Exclusion Proof:

- For Pi to enter the CS, `flag[j] == false` or `turn == i` must hold
- If both processes are in the CS, then `flag[0] == flag[1] == true`
- Since `turn` can only be 0 or 1, simultaneous entry is impossible

2. Progress Proof:

- If Pj does not want to enter the CS, `flag[j] == false` → Pi can enter
- If Pj sets `flag[j] = true`, then `turn` determines which one enters

3. Bounded Waiting Proof:

- While Pi is waiting, if Pj exits the CS and sets `flag[j] = false`, Pi enters
- If Pj wants to re-enter, it must set `turn = i`, so Pi enters first
- At most **1** additional entry is allowed (bounded)

Peterson's Solution – Limitations on Modern Architectures

Modern processors/compilers can **reorder instructions**

```
/* Peterson's entry section */
flag[i] = true;     /* ① */
turn = j;           /* ② */
```

- If the order of ① and ② is swapped, mutual exclusion can be broken
- Example: if `turn = j` executes first → both processes can enter the CS

```
Process 0                Process 1
turn = 1                 turn = 0, flag[1] = true
flag[0] = true
   CS                       CS       ← simultaneous entry!
```

- Solution: use **Memory Barriers** to enforce ordering

Part 3

Hardware Support for Synchronization (Ch 6.4)

Memory Barriers (Memory Fences)

Memory model classification:

- **Strongly ordered**: memory modifications by one processor are immediately visible to other processors
- **Weakly ordered**: modifications may not be immediately visible

Memory Barrier: an instruction that enforces the ordering of memory operations

```
/* Thread 1 */                    /* Thread 2 */
while (!flag)                     x = 100;
    memory_barrier();             memory_barrier();
print x;  /* guaranteed to print 100 */    flag = true;
```

- All loads/stores before the barrier complete before those after the barrier
- Peterson's Solution can work on modern hardware with appropriate memory barriers inserted
- A very **low-level** operation, primarily used by kernel developers

Hardware Instructions – test_and_set

test_and_set: an instruction that atomically tests and sets a value

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;          /* return the old value */
}
```

Mutual exclusion implementation using test_and_set:

```
/* lock is initialized to false */
do {
    while (test_and_set(&lock))
        ;               /* busy wait */

    /* critical section */

    lock = false;

    /* remainder section */
} while (true);
```

- Even if two instructions execute simultaneously, they are processed **sequentially** (atomic)

Hardware Instructions – compare_and_swap (CAS)

compare_and_swap (CAS): atomically compare and swap

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;       /* always returns the original value */
}
```

Mutual exclusion using CAS:

```
/* lock is initialized to 0 */
while (true) {
    while (compare_and_swap(&lock, 0, 1) ≠ 0)
        ;              /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}
```

CAS – Ensuring Bounded Waiting

Basic CAS does not guarantee bounded waiting. An algorithm that does:

```
/* Shared variables: boolean waiting[n] = {false}, int lock = 0 */
while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1);
    waiting[i] = false;

    /* critical section */

    j = (i + 1) % n;
    while ((j ≠ i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = 0;            /* no waiters */
    else
        waiting[j] = false;  /* yield to the next waiter */

    /* remainder section */
}
```

Atomic Variables

A high-level tool based on CAS that manipulates variables **without race conditions**

CAS-based `increment` function:

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;
    } while (temp ≠ compare_and_swap(v, temp, temp + 1));
}
```

Linux kernel examples:

```
atomic_t counter;
atomic_set(&counter, 5);      /* counter = 5 */
atomic_add(10, &counter);     /* counter = 15 */
atomic_sub(4, &counter);      /* counter = 11 */
atomic_inc(&counter);         /* counter = 12 */
int val = atomic_read(&counter);  /* val = 12 */
```

- **Note**: atomic variables are only effective for single variable updates
- Compound operations like bounded-buffer require mutexes or semaphores

Part 4

Mutex Locks (Ch 6.5)

Mutex Locks

The simplest synchronization tool (mutex = **mut**ual **ex**clusion)

```
acquire() {
    while (!available)
        ;               /* busy wait */
    available = false;
}

release() {
    available = true;
}
```

Usage pattern:

```
acquire(lock);
    /* critical section */
release(lock);
```

- `acquire()` and `release()` must be executed **atomically** (CAS-based)
- Lock state is managed with a **Boolean variable** `available`

Spinlock and Busy Waiting

- **Spinlock**: a mutex lock that uses busy waiting
  - The process keeps **spinning** (looping) while waiting for the lock
- **Disadvantage**: wastes CPU cycles (problematic in multiprogramming)
- **Advantage**: no context switch overhead
  - Two context switches are needed: transition to waiting state + return upon lock acquisition
  - Spinning is efficient when the lock is held for a **short time**
- Especially useful in **multicore systems**
  - One core spins while another core executes the CS
  - Rule of thumb: use a spinlock when holding time is **shorter than two context switches**
- **Lock contention**: contended (blocked) vs uncontended (available)
  - High contention → performance degradation

Part 5

Semaphores (Ch 6.6)

Semaphores – Definition

Only two **atomic operations** are allowed on an integer variable S (proposed by Dijkstra)

wait(S) – P operation (proberen, "to test"):

```
wait(S) {
    while (S ≤ 0)
        ;               /* busy wait */
    S--;
}
```

signal(S) – V operation (verhogen, "to increment"):

```
signal(S) {
    S++;
}
```

- **Modifications to S in wait() and signal() must be executed atomically**
- The test of S (`S ≤ 0`) and the modification (`S--`) must also be executed **without interruption**

Semaphore Types and Usage

Counting Semaphore:

- No restriction on the range of values (non-negative integers)
- **Manages a finite number of resources** (N resources → initialize S = N)
- Use resource: `wait(S)` → S decreases, Release resource: `signal(S)` → S increases
- If S = 0, all resources are in use → subsequent requests block

Binary Semaphore:

- Value can only be 0 or 1
- Behaves **similarly to a mutex lock**

Execution Order Control:

- Initialize `synch = 0`
- P1: `S1; signal(synch);` / P2: `wait(synch); S2;`
- S2 is guaranteed to execute after S1

Semaphore – Eliminating Busy Waiting

Busy waiting wastes CPU → solved using a **waiting queue**

```
typedef struct {
    int value;
    struct process *list;  /* waiting queue */
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        /* add this process to S->list */
        sleep();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value ≤ 0) {
        /* remove process P from S->list */
        wakeup(P);
    }
}
```

- If `S->value` is **negative**: its absolute value = number of waiting processes

Atomicity of Semaphore Implementation

- `wait()` and `signal()` must be executed **atomically**
- **Single processor**: solved by inhibiting interrupts
  - No other process can intervene during wait/signal execution
- **Multiprocessor**: disabling interrupts on all cores is inefficient
  - **CAS** or **spinlock** is used to ensure atomicity
- Busy waiting is not completely eliminated
  - Busy waiting occurs only in the short critical section inside wait/signal
  - For the application's long CS, a **sleep/wakeup** approach is used

Part 6

Monitors (Ch 6.7)

Difficulty of Using Semaphores

Incorrect usage of semaphores causes timing errors:

- **Order reversal**: `signal(mutex) → CS → wait(mutex)`
  - Multiple processes can enter the CS simultaneously → mutual exclusion violation
- **Duplicate wait**: `wait(mutex) → CS → wait(mutex)`
  - Permanently blocked at the second wait → **deadlock**
- **Omitting wait or signal**:
  - Mutual exclusion violation or permanent blocking
- Such errors **only occur under specific execution orders** → very difficult to debug
- Solution: use a **high-level synchronization construct** (Monitor)

Monitor – Structure

Monitor: an ADT that automatically provides mutual exclusion for its internal functions

```
monitor monitor_name {
    /* shared variable declarations */

    function P1( ... ) {  ...  }
    function P2( ... ) {  ...  }
     ...
    function Pn( ... ) {  ...  }

    initialization_code( ... ) {  ...  }
}
```

- **Only one process at a time** can be active inside the monitor
- Programmers do not need to explicitly write synchronization code
- Internal variables are accessible **only by local functions**
- Languages such as Java and C# support the monitor concept

Monitor – Condition Variables

Monitors alone are insufficient for some synchronization → **Condition Variables** are introduced

```
condition x, y;

x.wait();     /* the calling process is suspended */
x.signal();   /* resumes one waiting process */
```

- `x.signal()` has **no effect** if no process is waiting
  - Different from semaphore's signal (semaphore always changes the value)

Handling after signal:

- **Signal and wait**: the signaler (P) waits, the awakened process (Q) runs
- **Signal and continue**: P continues running, Q runs after P leaves the monitor
- **Compromise**: signal immediately leaves the monitor, Q resumes immediately

Implementing a Monitor with Semaphores

Signal-and-wait implementation:

```
/* Transform each external function F of the monitor as follows */
semaphore mutex = 1;     /* protects monitor entry */
semaphore next = 0;      /* for waiting after signal */
int next_count = 0;      /* number of processes waiting on next */

/* Body of function F */
wait(mutex);
    ... body of F ...
if (next_count > 0)
    signal(next);        /* wake up a waiting signaler if any */
else
    signal(mutex);       /* otherwise release the monitor lock */
```

Condition variable `x` implementation:

```
/* x.wait() */                    /* x.signal() */
x_count++;                        if (x_count > 0) {
if (next_count > 0)                   next_count++;
    signal(next);                      signal(x_sem);
else                                   wait(next);
    signal(mutex);                     next_count--;
```

Monitor – ResourceAllocator Example

A monitor that manages single resource allocation (using conditional-wait):

```
monitor ResourceAllocator {
    boolean busy;
    condition x;

    void acquire(int time) {
        if (busy)
            x.wait(time);    /* use time as priority */
        busy = true;
    }

    void release() {
        busy = false;
        x.signal();          /* resume the process with the smallest time value */
    }

    initialization_code() {
        busy = false;
    }
}
```

- `x.wait(c)` : stores priority number c, `x.signal()` resumes the process with the smallest c

Part 7

Liveness (Ch 6.8)

Deadlock

A state where two or more processes are waiting for a signal that only the other can produce

```
P0              P1
wait(S);        wait(Q);
wait(Q);        wait(S);
 ...             ...
signal(S);      signal(Q);
signal(Q);      signal(S);
```

- P0 acquires S, P1 acquires Q → they wait for each other forever
- **Deadlocked state**: every process in the set is waiting for an event that only another process in the set can cause
- Deadlock primarily occurs during **resource acquisition/release**
- Covered in detail in Ch 8 (Prevention, Avoidance, Detection, Recovery)

Priority Inversion

A phenomenon where a lower-priority process indirectly blocks a higher-priority process

- Three processes: L (low), M (medium), H (high)
- L holds semaphore S → H waits for S → M preempts L
- Result: H has higher priority than M, but is delayed because of M

Solved with Priority Inheritance Protocol:

- Temporarily elevate L's priority to H's priority
- M cannot preempt L → L quickly releases S
- After releasing the lock, L reverts to its original priority

Real-world case: Mars Pathfinder (1997) – repeated system resets due to priority inversion in VxWorks RTOS, resolved by enabling priority inheritance

Evaluating Synchronization Tools

CAS-based (Optimistic) vs Locking (Pessimistic):

Contention Level	CAS-based	Traditional Locking
Uncontended	Fast (slight advantage)	Fast
Moderate	Much faster	Slow (context switch cost)
High	Slow (repeated retries)	Fast (queue-based)

- **Low-level**: Hardware instructions (CAS, test_and_set) – primarily used as the foundation for other tools
- **Mid-level**: Mutex locks, Semaphores – most commonly used in kernel/applications
- **High-level**: Monitors, Condition variables – language-level support

Part 8

Classic Problems of Synchronization (Ch 7.1)

Bounded-Buffer Problem

Shared buffer (size n), **Producer** and **Consumer**

Semaphore usage:

- `mutex = 1` (protects buffer access, binary semaphore)
- `empty = n` (number of empty slots)
- `full = 0` (number of filled slots)

Shared data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0;
```

- `empty` and `full` are **counting semaphores**
- `mutex` is a **binary semaphore** (mutual exclusion)

Bounded-Buffer – Producer/Consumer Code

```
/* Producer */
while (true) {
    /* produce an item */

    wait(empty);
    wait(mutex);

    /* add item to buffer */

    signal(mutex);
    signal(full);
}
```

```
/* Consumer */
while (true) {
    wait(full);
    wait(mutex);

    /* remove item from buffer */
    /* to next_consumed */

    signal(mutex);
    signal(empty);

    /* consume next_consumed */
}
```

Key Points:

- The order of `wait` calls matters: **counting semaphore first, then mutex**
- If the order is reversed (`wait(mutex)` → `wait(empty)`), **deadlock** can occur
  - When the buffer is full, the producer holds mutex and waits on empty → the consumer also cannot acquire mutex
- Note the **symmetric structure** of Producer and Consumer

Readers-Writers Problem

Readers (read only) and **Writers** (read + write) exist for shared data

- **Multiple readers** can access simultaneously
- **A writer requires exclusive access** (only one at a time)

First Readers-Writers Problem (readers priority):

- When a reader is waiting, it is allowed to enter even if a writer is waiting
- **Writer starvation** is possible

Second Readers-Writers Problem (writers priority):

- When a writer is waiting, new readers are not allowed to enter
- **Reader starvation** is possible
- Starvation is possible regardless of the approach
- Reader-Writer Lock: distinguishes read mode (shared) / write mode (exclusive)

Readers-Writers – First Problem Solution

```
/* Shared variables */
semaphore rw_mutex = 1;     /* reader-writer mutual exclusion */
semaphore mutex = 1;         /* protects read_count */
int read_count = 0;
```

```
/* Writer */
while (true) {
    wait(rw_mutex);

    /* writing is performed */

    signal(rw_mutex);
}
```

```
/* Reader */
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    /* reading is performed */

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}
```

- The first reader locks `rw_mutex`, and the last reader releases it

Reader-Writer Locks

A generalized synchronization tool for the Readers-Writers problem

- **Read mode**: multiple threads can acquire simultaneously
- **Write mode**: only one thread can acquire (exclusive)
- Useful situations:
  - Applications where read/write can be easily distinguished
  - Applications where **readers outnumber writers** (overhead is compensated by concurrency)
- POSIX: `pthread_rwlock_t`
- Java: `ReentrantReadWriteLock`
- Windows: Slim Reader-Writer Lock

Dining-Philosophers Problem

5 philosophers sit at a round table and must **pick up both adjacent chopsticks** to eat

- Philosophers alternate between **thinking** and **eating**
- Both the left and right chopsticks must be available to start eating
- Only one chopstick can be picked up at a time
- After finishing eating, both chopsticks are put down
- **A classic concurrency control problem**
  - The problem of allocating multiple resources to multiple processes in a deadlock-free, starvation-free manner

Dining-Philosophers – Semaphore Solution

Each chopstick is represented as a semaphore:

```
semaphore chopstick[5];  /* all initialized to 1 */

/* Philosopher i */
while (true) {
    wait(chopstick[i]);              /* left chopstick */
    wait(chopstick[(i + 1) % 5]);    /* right chopstick */

    /* eat for a while */

    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);

    /* think for a while */
}
```

Problem: if all 5 philosophers pick up their left chopstick simultaneously → **deadlock!**

- All chopstick values become 0 → no one can pick up the right chopstick

Dining-Philosophers – Deadlock Solutions

Method 1: limit to at most **4** philosophers sitting at the table simultaneously

- Controlled with an additional semaphore `seats = 4`

Method 2: only pick up chopsticks when both sides are available

- Check and pick up both chopsticks within a critical section
- The monitor solution implements this approach

Method 3: **Asymmetric** approach

- Odd-numbered philosophers: left first → right
- Even-numbered philosophers: right first → left
- Breaks the **circular wait** condition → prevents deadlock
- Note: deadlock-free does not guarantee starvation-free

Dining-Philosophers – Monitor Solution

Ensures that a philosopher eats only when both chopsticks are available:

```
monitor DiningPhilosophers {
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] ≠ EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);    /* check left neighbor */
        test((i + 1) % 5);    /* check right neighbor */
    }

    void test(int i) {
        if (state[(i + 4) % 5] ≠ EATING &&
            state[i] = HUNGRY &&
            state[(i + 1) % 5] ≠ EATING) {
            state[i] = EATING;
```

Dining-Philosophers – Monitor Usage

Each philosopher calls the monitor in the following order:

```
DiningPhilosophers.pickup(i);
    ...
    /* eat */
    ...
DiningPhilosophers.putdown(i);
```

How it works:

- `pickup(i)` : changes state to HUNGRY, then calls `test(i)`
  - If neither neighbor is EATING → changes to EATING
  - Otherwise, `self[i].wait()` → suspended until a neighbor calls putdown
- `putdown(i)` : changes state to THINKING, then calls `test()` for both neighbors
  - If a neighbor is HUNGRY and conditions are met → calls `self[j].signal()` for that neighbor
- **No deadlock**: eating only when both sides are available
- **Starvation is still possible** (exercise)

Part 9

Synchronization in the Kernel (Ch 7.2)

Windows Synchronization

Single-processor: protects global resources via interrupt masking

Multiprocessor: uses **spinlocks** (only for short code segments)

- Never preempted while holding a spinlock

Dispatcher Objects (thread synchronization outside the kernel):

- **Mutex lock**, **Semaphore**, **Event**, **Timer**
- States: **signaled** (available) / **nonsignaled** (unavailable)
- Acquiring a nonsignaled object → thread blocked → placed in waiting queue
- When transitioning to signaled → waiting thread is moved to ready state

Critical-Section Object (user-mode mutex):

- First attempts with a spinlock → if waiting too long, allocates a kernel mutex
- Can acquire/release without kernel intervention → very efficient
- Significant performance benefit when contention is low

Linux Synchronization

Linux has been a **fully preemptive kernel** since 2.6 (previously nonpreemptive)

Tool	Purpose	Characteristics
`atomic_t`	Atomic integer operations	Counters, sequence generators
`spinlock_t`	Short CS protection (SMP)	Replaced with preempt disable on single CPU
`mutex`	Long CS protection	Sleep-based lock (sleeping lock)
`semaphore`	Counting semaphore	Suitable for long waits

```
/* Linux spinlock/preemption mapping */
/* Single Processor        | Multiple Processors    */
/* Disable kernel preempt  | Acquire spin lock      */
/* Enable kernel preempt   | Release spin lock      */
```

- Spinlocks and mutexes are **nonrecursive** (acquiring the same lock twice causes deadlock)

POSIX Synchronization

API available at the user level (UNIX, Linux, macOS)

POSIX Mutex Locks:

```
#include <pthread.h>
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

pthread_mutex_lock(&mutex);
    /* critical section */
pthread_mutex_unlock(&mutex);
```

POSIX Semaphores (Named):

```
#include <semaphore.h>
sem_t *sem;
sem = sem_open("SEM", O_CREAT, 0666, 1);

sem_wait(sem);
    /* critical section */
sem_post(sem);
```

- Named semaphore: can be shared among multiple **unrelated processes**

POSIX Condition Variables

Since C has no monitors, condition variables are used together with mutexes

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;
pthread_mutex_init(&mutex, NULL);
pthread_cond_init(&cond_var, NULL);
```

Waiting thread:

```
pthread_mutex_lock(&mutex);
while (a ≠ b)
    pthread_cond_wait(&cond_var, &mutex);   /* release mutex + suspend */
pthread_mutex_unlock(&mutex);
```

Signaling thread:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);   /* wake up one waiting thread */
pthread_mutex_unlock(&mutex);
```

- `pthread_cond_wait()` **automatically releases the mutex** and waits

Part 10

Java Synchronization (Ch 7.4)

Java Monitors

Java provides **monitor-like** synchronization at the language level

```
public class BoundedBuffer<E> {
    private static final int BUFFER_SIZE = 5;
    private int count, in, out;
    private E[] buffer;

    public synchronized void insert(E item) {
        while (count == BUFFER_SIZE) {
            try { wait(); }
            catch (InterruptedException ie) { }
        }
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        notify();
    }

    public synchronized E remove() {
        while (count == 0) {
            try { wait(); }
            catch (InterruptedException ie) { }
        }
        E item = buffer[out];
```

Java Monitors – How They Work

- The `synchronized` keyword: automatically acquires the **object lock** when the method is called
- Every Java object has **one lock**, an **entry set**, and a **wait set**

When wait() is called:

1. Releases the object lock
2. Changes the thread state to blocked
3. Places the thread in the **wait set**

When notify() is called:

1. Selects an arbitrary thread T from the wait set
2. Moves T to the **entry set**
3. Changes T's state to runnable
4. When T reacquires the lock, it returns from wait()

- `notifyAll()`: moves **all** threads from the wait set to the entry set

Java ReentrantLock & Semaphore

ReentrantLock: a more flexible locking mechanism than synchronized

```
Lock key = new ReentrantLock();
key.lock();
try {
    /* critical section */
} finally {
    key.unlock();    /* must release in finally */
}
```

- Supports a **fairness parameter** (prioritizes the thread that has waited the longest)
- Reentrant: a thread that already holds the lock can lock it again

Semaphore:

```
Semaphore sem = new Semaphore(1);
try {
    sem.acquire();
    /* critical section */
} catch (InterruptedException ie) { }
finally {
```

Java Condition Variables

Named condition variables allow waiting on **specific conditions**

```
Lock lock = new ReentrantLock();
Condition[] condVars = new Condition[5];
for (int i = 0; i < 5; i++)
    condVars[i] = lock.newCondition();

public void doWork(int threadNumber) {
    lock.lock();
    try {
        if (threadNumber ≠ turn)
            condVars[threadNumber].await();

        /* do some work */

        turn = (turn + 1) % 5;
        condVars[turn].signal();
    } catch (InterruptedException ie) { }
    finally {
        lock.unlock();
    }
}
```

- `await()` : releases the lock + suspends (similar to wait())

Lab

Dining-Philosophers Implementation

Lab – Dining-Philosophers with pthreads

Basic implementation using mutexes:

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#define N 5

pthread_mutex_t chopstick[N];

void *philosopher(void *arg) {
    int id = *(int *)arg;
    while (1) {
        printf("Philosopher %d is thinking\n", id);
        usleep(rand() % 1000000);

        pthread_mutex_lock(&chopstick[id]);
        pthread_mutex_lock(&chopstick[(id + 1) % N]);

        printf("Philosopher %d is eating\n", id);
        usleep(rand() % 1000000);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id + 1) % N]);
```

Lab – Deadlock Solution: Asymmetric Approach

Even-numbered philosophers pick up the **right first**, odd-numbered pick up the **left first**:

```
void *philosopher_asymmetric(void *arg) {
    int id = *(int *)arg;
    int first, second;

    if (id % 2 == 0) {
        first = (id + 1) % N;    /* right first */
        second = id;
    } else {
        first = id;               /* left first */
        second = (id + 1) % N;
    }

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        usleep(rand() % 1000000);

        pthread_mutex_lock(&chopstick[first]);
        pthread_mutex_lock(&chopstick[second]);

        printf("Philosopher %d is eating\n", id);
        usleep(rand() % 1000000);
```

Lab – POSIX Condition Variable-Based Monitor Implementation

Implementing the monitor solution with the POSIX API:

```
#include <pthread.h>
#define N 5

enum {THINKING, HUNGRY, EATING} state[N];
pthread_mutex_t monitor_lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t self[N];

void test(int i) {
    if (state[(i+4)%N] ≠ EATING && state[i] == HUNGRY
        && state[(i+1)%N] ≠ EATING) {
        state[i] = EATING;
        pthread_cond_signal(&self[i]);
    }
}

void pickup(int i) {
    pthread_mutex_lock(&monitor_lock);
    state[i] = HUNGRY;
    test(i);
    while (state[i] ≠ EATING)
```

Summary

Key concepts this week:

- **Race Condition** and the 3 requirements of the **Critical-Section Problem** (Mutual Exclusion, Progress, Bounded Waiting)
- **Peterson's Solution**: software-based, not guaranteed to work on modern hardware due to instruction reordering
- **Hardware Support**: Memory Barrier, test_and_set, compare_and_swap, Atomic Variables
- **Mutex Lock**: acquire/release, trade-off of Spinlock (busy waiting)
- **Semaphore**: counting vs binary, eliminating busy waiting (waiting queue), wait/signal
- **Monitor**: high-level synchronization, Condition Variable (wait/signal), signal-and-wait vs signal-and-continue
- **Liveness**: Deadlock, Priority Inversion, Priority Inheritance Protocol
- **Classic Problems**: Bounded-Buffer, Readers-Writers, Dining-Philosophers
- **OS-specific synchronization**: Windows (Dispatcher Objects), Linux (atomic_t, spinlock, mutex), POSIX (pthread_mutex, sem_t, pthread_cond)
- **Java Synchronization**: synchronized, wait/notify, ReentrantLock, Condition

Next Week Preview

Week 10 – Deadlocks (Ch 8)

- Four necessary conditions for deadlock (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait)
- Resource-Allocation Graph
- Deadlock Prevention
- Deadlock Avoidance (Banker's Algorithm)
- Deadlock Detection
- Deadlock Recovery